

PROPOSED – AWAITING REVIEW

git-mcp: reaching any repo, on any tailnet node, in plain English

A third remote-capable MCP server, alongside docker-mcp and postgres-mcp — same gateway, same tailnet, same "point it anywhere" idea, but git's own shape forces a genuinely different design underneath. Nothing in this doc has been built yet.

status: not implemented – plan only 2026-08-16

00 What's already solved, and what isn't

"Plain English instead of git commands" needs nothing new. That's what MCP + an LLM already does — "list containers on katmint" already becomes a structured `list-containers(docker_host= ... )` call today, with no NLP layer inside docker-mcp itself. Whatever git-mcp exposes as typed tools gets the same treatment automatically.

"Point it at any tailnet node" is the real design problem — and git isn't like Docker or Postgres here. Docker has a native remote-daemon protocol (`docker -H ssh:// ...`); Postgres has a native network protocol (any client, any connection string). Git's working-tree commands — `status`, `diff`, `commit` — are inherently local filesystem operations. There's no "connection string" that gives you `git status` on a different machine's repo. The rest of this doc is about how to bridge that gap safely.

01 Decisions already made

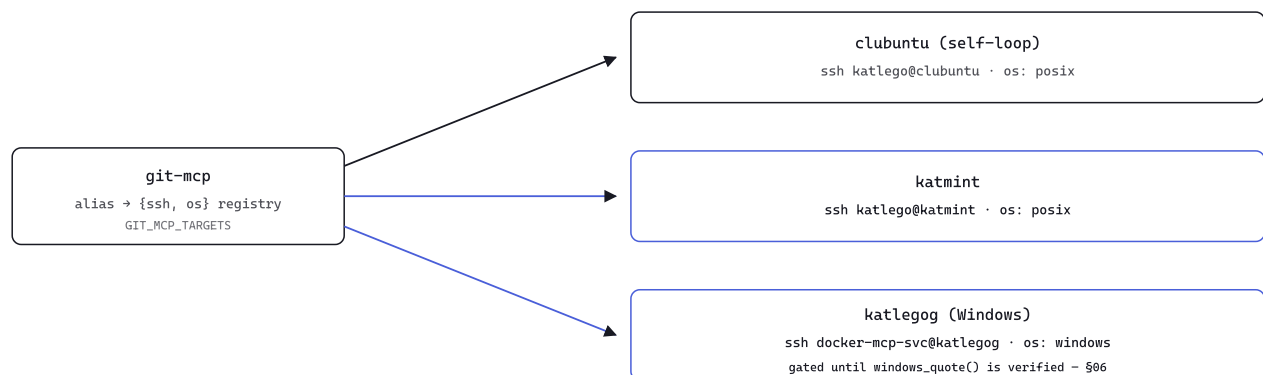
QUESTION	DECISION	WHY
Remote access mechanism	SSH-execute	Mirrors docker-mcp exactly; full command set available, not just sync operations
Tool shape	Typed tools per operation	Matches docker-mcp/postgres-mcp — neither exposes a raw passthrough tool; the tool <i>set</i> is the safety boundary
Repo scope	Arbitrary path per call	More flexible than a fixed registry — but raises the stakes on argument handling (§03)

02 Target registry, not a free-form host string

Every tool takes a required `target` naming a server-side-registered alias — `clubuntu`, `katmint`, `katlegog` — never a raw `ssh://user@host` string the way `docker-mcp`'s `docker_host` works. Two reasons this is the better shape specifically for `git-mcp`:

- ▶ Closes an SSH option-injection edge case a free-form string allows: a hostname beginning with `-` can be read as an `ssh` flag, not a destination. An alias can never do that.
- ▶ Gives a fixed place to record each target's *shell dialect* — critical because `katlegog` is Windows (see §03).

No "local" default exists. Unlike `docker-mcp`, which has a real local Docker daemon reachable via a mounted socket, `git-mcp`'s own container has no meaningful repos on it — so even "target `clubuntu`" goes over SSH, a self-loop entry in the same registry as every other host.

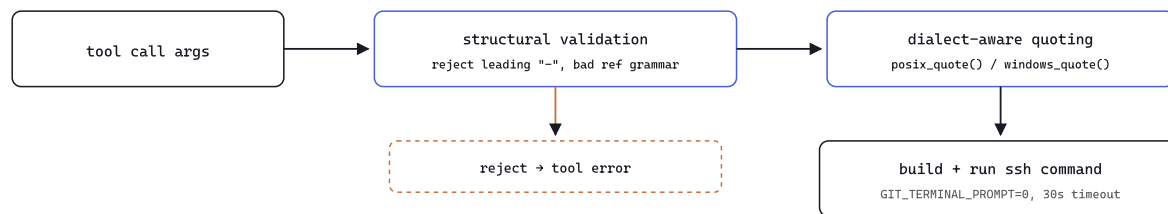


A dedicated `git_mcp_ed25519` keypair, separate from `docker-mcp`'s own, appended as a second `authorized_keys` line on the same already-provisioned accounts (no new OS accounts needed).

03 The hard problem: safe remote command construction

Every tool ultimately runs `ssh <destination> "git -C <repo_path> <args ... >"`. SSH's protocol always re-joins the remote command into one string and hands it to the target's login shell — there is no argv-preserving remote-exec mode. That creates *two independent risks*, and both need addressing:

LAYER	RISK	MITIGATION
1. Shell-metacharacter injection	A backtick or <code>\$()</code> in a commit message gets executed by the target's shell instead of stored as text	Dialect-aware quoting: <code>shlex.quote()</code> for POSIX targets, a separate <code>windows_quote()</code> for <code>cmd.exe</code> — never assume one dialect , select via the target registry's <code>os</code> field
2. Argument-shape / flag injection	A perfectly shell-safe string can still be dangerous by <i>position</i> — a branch name of <code>--upload-pack=...</code> or a pathspec of <code>--force</code> gets parsed by <code>git</code> itself as a flag once it arrives intact	Structural validation <i>before</i> quoting: reject any ref/branch/remote/pathspec starting with <code>-</code> , validate against git's ref-name grammar



Both layers live in one module (`quoting.py`), called from one place (`ssh_executor.py`) — mirrors how `docker_executor.py` centralizes this today.

Neither layer alone is sufficient — quoting stops shell re-splitting, validation stops git from parsing a clean argument as a flag.

04 v1 tool list

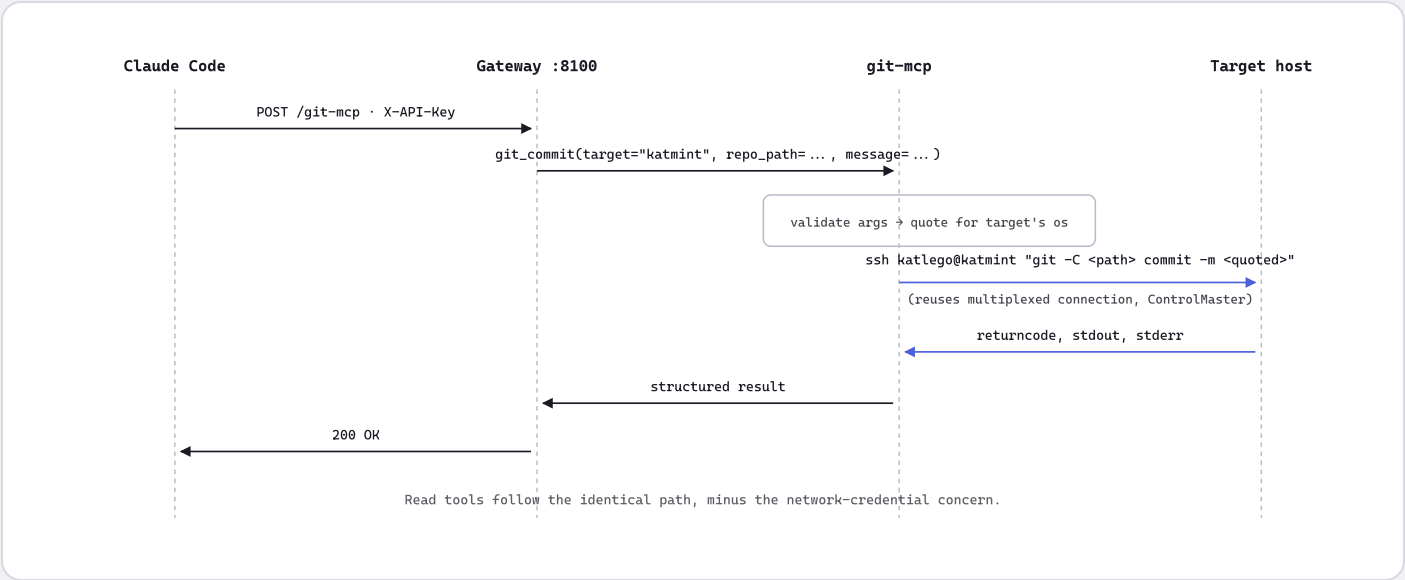
CATEGORY	TOOLS
Read	<code>git_status</code> , <code>git_log</code> , <code>git_diff</code> , <code>git_show</code> , <code>git_branch_list</code> , <code>git_remote_list</code>
Safe mutating	<code>git_add</code> , <code>git_commit</code> (no <code>--amend</code>), <code>git_fetch</code> , <code>git_pull</code> (plain merge, no <code>--rebase</code>), <code>git_checkout</code> (switch or <code>-b</code> create only), <code>git_branch_create</code> , <code>git_push</code> (no <code>force</code> parameter anywhere in the schema)
Excluded from v1	<code>force-push</code> , <code>reset --hard</code> , <code>clean -f / -fdx</code> , <code>rebase</code> , <code>commit --amend</code> , history rewriting, force branch delete, <code>tag -f</code> , <code>checkout --force</code> , any raw passthrough command tool

Same rule `docker-mcp/postgres-mcp` already follow: the tool set itself is the allowlist. Nothing here is gated behind a runtime `confirm=true` flag, because nothing in this codebase works that way — dangerous operations are simply never implemented.

`GIT_TERMINAL_PROMPT=0` on every network-touching call (`fetch/pull/push`) — without it, a target needing interactive

git credentials hangs the MCP call forever with no TTY to prompt against. Paired with a 30s subprocess timeout.

05 One call, end to end



Same four-hop shape as docker-mcp's own remote calls — validation and quoting are the two extra steps git-mcp adds inside its own box.

06 Staged rollout: POSIX first, Windows gated

Implement and fully verify `clubuntu` + `katmint` (both POSIX, `shlex.quote()` covers them) before enabling the `katlegog` registry entry. `windows_quote()` gets its own explicit test cases — `cmd.exe` metacharacters `& | < > ^ %` — before that target goes live. Don't ship an unverified Windows quoting path silently alongside the proven POSIX one.

07 File-by-file changes

FILE	CHANGE
<code>git-mcp/</code> (new)	Hybrid layout: docker-mcp's SSH transport + postgres-mcp's FastMCP/ <code>tools/</code> package shape. <code>Dockerfile</code> , <code>ssh_config</code> , <code>pyproject.toml</code> , <code>mcp_server.py</code> , <code>src/git_mcp/{config,quoting,ssh_executor,server}.py</code> , <code>src/git_mcp/tools/{_scope,read,write}.py</code>
<code>docker-compose.yml</code> / <code>.deploy.yml</code>	New <code>git-mcp</code> service block; <code>GIT_MCP_UPSTREAM_URL</code> + <code>git-mcp</code> added to <code>gateway</code>

FILE	CHANGE
<code>.env.example</code>	New <code>--- git-mcp ---</code> section: keypair setup steps, <code>GIT_MCP_SSH_KEY_PATH</code> , <code>GIT_MCP_SSH_KNOWN_HOSTS_PATH</code> , <code>GIT_MCP_TARGETS</code> registry
<code>api/main.py</code>	<code>proxy_git_mcp</code> , identical shape to <code>proxy_docker_mcp</code> , mounted at <code>/git-mcp{path:path}</code>
<code>.github/workflows/build.yml</code>	Throwaway <code>/dev/null</code> values for the two new bind-mount-source vars, <code>'{}'</code> for the registry
<code>README.md</code>	New services-table row, registration command, two new <code>Known limitations</code> bullets

Reused directly, not re-derived: `docker-mcp/src/docker_mcp/docker_executor.py` (SSH mechanics template), `postgres-mcp/src/postgres_mcp/tools/_scope.py` (per-call-override param shape), `docker-mcp/ssh_config` (near-verbatim, just repoint `IdentityFile`), and the Windows admin-account SSH pitfall already solved for `docker-mcp` (`docker-mcp-svc` in `docker-users`, not `Administrators`) — not rediscovered.

08 Verification plan

- ▶ **Read tools against real repos** (safe, no mutation possible): this repo itself, on katlegog and its clone on clubuntu — confirms Windows-path handling even for read-only calls.
- ▶ **Write tools against a disposable scratch repo only** — never this repo, never a real upstream remote. Bootstrap a bare repo + clone manually, exercise `git_add` / `git_commit` / `git_push` against it.
- ▶ **Deliberate injection test cases, not an afterthought:** a commit message with shell metacharacters (should pass through as inert data), a branch name starting with `-` (should be rejected by `validate_ref_name`) — repeated with `cmd.exe` metacharacters once `windows_quote()` exists.
- ▶ Confirm `GIT_TERMINAL_PROMPT=0` actually prevents a hang against a credential-requiring remote.
- ▶ Same deploy pipeline as `docker-mcp`: push → build, tag → deploy, confirm mounts via `docker inspect`, then test through the real `mcp__git__*` tools.

09 Known limitation, flagged not solved

`push` / `pull` / `fetch` against a real upstream (e.g. GitHub) depends on the *target account's own* git credentials (e.g. `docker-mcp-svc`'s own SSH key or token) – a separate concern from the `git-mcp`→host SSH tunnel itself. Verification sidesteps this with a disposable bare-repo remote; making real-remote push/pull work for a given account is a follow-up, not part of this plan.

`git-mcp` – design plan, awaiting review. Nothing in this document has been implemented.